

■■ DeepShield AI

Deepfake Image Detection - Project Documentation & Report

1. Agentic Development Tools Used

During pair programming with the user, I utilized the following specialized agentic tools to construct, execute, and verify the deepfake image detection codebase:

Agent Tool	Description & Purpose in this Project
list_dir	Scanned the initial empty workspace to map out files and subdirectories.
write_to_file	Created the original python modules, configuration files (requirements.txt), planning documents, and the README manual.
replace_file_content	Performed contiguous edits. Used to modify code bugs and adapt train.py, predict.py, and app.py to Keras 3 fallbacks.
view_file	Read existing codebase files line-by-line to target precise replacements and ensure code integrity.
run_command	Executed terminal scripts to create the Python 3.10 virtual environment, install requirements, run training, and test predictions.
manage_task	Monitored background execution status and sent terminal keyboard inputs (newline) to bypass Streamlit email registration prompts.
browser_subagent	Navigated to the live Streamlit URL, verified HTML components, rendered glassmorphic styles, took screenshots, and recorded interactions.

2. System Architecture & Module Descriptions

A. `dataset_loader.py` (Data Pipelines)

B. `preprocessing.py` (Face Cropping & Augmentation)

C. `model.py` (Transfer Learning CNN)

D. `utils.py` (Headless Visualization)

E. train.py & Keras 3 Legacy H5 Fallback Strategy

3. Setup and Execution Manual

Step 1: Environment Setup

```
py -3.10 -m venv venv  
.\venv\Scripts\activate  
pip install -r requirements.txt
```

Step 2: Train the Model

```
python train.py --epochs 20 --batch_size 16
```

Step 3: Run Command Line Inference

```
python predict.py --image dataset/real/real_000.jpg --model  
saved_model/deepfake_detector_best.keras
```

Step 4: Launch the GUI App

```
streamlit run app.py
```